SIMATS ENGINEERING
CO3-ASSESSMENT TOOL1- CASE STUDY

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS: 4 **Total Marks:40**

Code of Conduct:

I KAVIYA.G(192524281*) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kaviya.G

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu
Ii. If a patient has Fever AND Rash → Possible Measles
Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia
Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goalAssignment Answers — Logical Agents and Reasoning
Case Study 1 — Smart Medical Diagnosis System

(a) Representation using Propositional Logic

Let the following propositional symbols represent the patient's conditions:

Symbol	**Meaning**
F	Patient A has Fever
C	Patient A has Cough
B	Patient A has Breathlessness
Flu	Patient A possibly has Flu
M	Patient A possibly has Measles
P	Patient A possibly has Pneumonia

Rules

Introduction

A hospital is developing a rule-based logical agent to assist doctors during the preliminary stage of patient diagnosis. The system stores symptoms as facts and uses predefined rules to identify possible diseases. The system does not replace a doctor; it simply applies logical reasoning to the information available in its knowledge base.

For Patient A, the available information is:

- Fever = True
- Cough = True
- Breathlessness = True
-

The system contains rules for identifying possible Flu, Measles and Pneumonia.

Representation using Propositional Logic

Propositional Logic represents statements using symbols that can have either a True or False value.

Step 1 – Define the symbols

Symbol	Meaning
F	Patient has Fever
C	Patient has Cough
B	Patient has Breathlessness

R	Patient has Rash
FL	Patient may have Flu
M	Patient may have Measles
P	Patient may have Pneumonia

The logical operators used are:

- $\wedge$ – AND
- $\vee$ – OR
- $\neg$ – NOT
- $\rightarrow$ – implies / if-then Step 2 – Represent the rules Rule I

"If a patient has Fever AND Cough, then Possible Flu." Therefore:

Step 3 – Represent the facts For Patient A:

F = True C = True B = True

Therefore, the knowledge base can be written as:

KB = $\{F, C, B, F \wedge C \rightarrow FL, F \wedge R \rightarrow M, C \wedge B \rightarrow P\}$

There is no fact stating that Patient A has a rash. Hence, R is not known to be true.

(b) Forward Chaining for Patient A Meaning of Forward Chaining

Forward Chaining is a data-driven reasoning technique. It begins with the facts that are already known and checks which rules can be applied. Whenever all the conditions of a rule are satisfied, its conclusion is added to the knowledge base.

In this case, we begin with:

Fever, Cough and Breathlessness

and move forward to obtain possible diagnoses.

Step 1 – Start with the known facts The initial facts are:

F = True C = True B = True Therefore:

$\{F, C, B\}$

is the initial set of facts.

Step 2 – Check Rule I Rule I is:

$F \wedge C \rightarrow FL$

The rule requires:

- Fever
- Cough Both are available. Therefore:

$F \wedge C$ = True

So the rule fires. New fact derived: FL = True Hence:

Patient A may have Flu.

Step 3 – Check Rule II Rule II is:

$F \wedge R \rightarrow M$

The rule requires:

- Fever
- Rash We know:

F = True
However, there is no fact stating: R = True
Therefore, the rule cannot be fire So:
Measles cannot be derived from the given information.
It is important not to assume that Rash is present simply because it was mentioned in the rule.

Step 4 – Check Rule III Rule III is:
$C \land B \rightarrow P$
The required conditions are: C = True
B = True
Both facts are available. Therefore, Rule III fires. New fact derived:
P = True Hence:
Patient A may have Pneumonia.
Forward Chaining Table

Step	Facts available	Rule checked	Result
Initial	F, C, B	—	Initial facts
1	F, C	$F \land C \rightarrow FL$	FL derived
2	F, no R	$F \land R \rightarrow M$	Cannot fire

Therefore, after forward chaining: Possible Flu = True
Possible Pneumonia = True Possible Measles = Not derived

Final Result of Forward Chaining
The logical agent identifies two possible conditions from the given rules:
Patient A → Possible Flu
Patient A → Possible Pneumonia
The result is based only on the facts supplied to the system.

(c) **Backward Chaining to Prove Pneumonia Meaning of Backward Chaining**
Backward Chaining works in the opposite direction from Forward Chaining. Instead of starting with the symptoms, it starts with the goal and asks: "What conditions must be true for this goal to be true?"
The goal given in the question is: Patient A has Pneumonia.
Representing the goal:
P

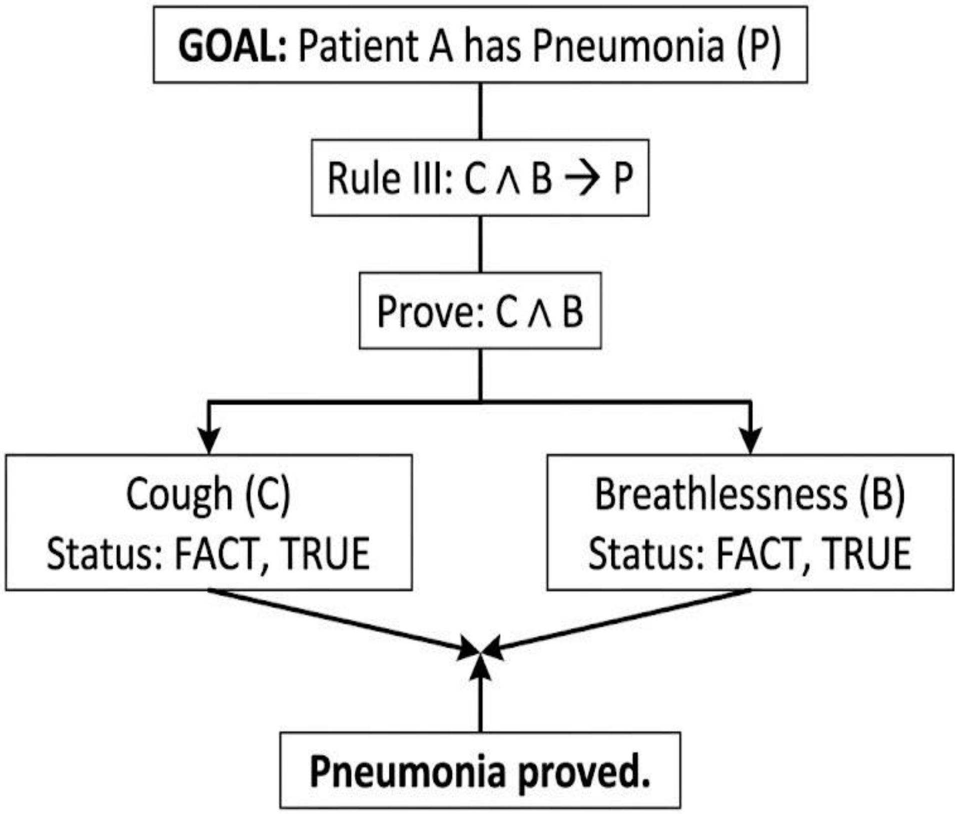
Step 1 – Find a rule that produces P The knowledge base contains:
$C \land B \rightarrow P$
Therefore, in order to prove: P
we must prove: $C \land A$
This gives two sub-goals:
1. Cough
2. Breathlessness

Step 2 – Verify Cough

The knowledge base contains: C = True
Therefore, the first sub-goal is satisfied.

Step 3 – Verify Breathlessness The knowledge base contains:
B = True
Therefore, the second sub-goal is also satisfied. Since both conditions are true:
$C \land B$ = True Therefore:
P = True

Goal Reduction Tree



Conclusion
The goal Patient A has Pneumonia is successfully proved using Backward Chaining because both required conditions, Cough and Breathlessness, are already present as facts.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. **∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]**
II. **∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]**
III. **∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]**
IV. **Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)**

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Introduction

A smart city uses an AI-based traffic management system to control traffic signals and provide a clear route for emergency vehicles such as ambulances.
Since the system deals with different objects and relationships between objects, First-Order Logic (FOL) is more suitable than simple propositional logic.
The knowledge base contains the following rules. Rule 1
∀x [Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)] Rule 2
∀x [ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x)] Rule 3
∀x∀y [Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)] Facts Vehicle(Ambulance) EmergencyType(Ambulance) Vehicle(CarA) Behind(CarA,Ambulance)

(a) **Unification of Rule 1 Meaning of Unification**

Unification is the process of finding substitutions for variables so that two logical expressions become identical.
Rule 1 is:
Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) The given facts are:
Vehicle(Ambulance) EmergencyType(Ambulance)
The variable x can be replaced by Ambulance. Therefore:
θ = {x/Ambulance}
This is the Most General Unifier (MGU). After applying the substitution:
Vehicle(Ambulance) ∧ EmergencyType(Ambulance) → ClearPath(Ambulance) Both conditions are available as facts.
Therefore: ClearPath(Ambulance) is derived.

Applying Rule 2
Rule 2:

ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) Using:
θ = {x/Ambulance}
we get:
ClearPath(Ambulance) → GreenSignal(Ambulance) ∧ ¬RedSignal(Ambulance) Since ClearPath(Ambulance) is true:
GreenSignal(Ambulance) and
¬RedSignal(Ambulance) are derived. Applying Rule 3
Rule 3 is:
Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) The required facts are:
Vehicle(CarA) Behind(CarA,Ambulance) GreenSignal(Ambulance) Therefore:
x = CarA
y = Ambulance The substitution is:
θ_2 = {x/CarA, y/Ambulance} After substitution:
Vehicle(CarA) ∧ Behind(CarA,Ambulance) ∧ GreenSignal(Ambulance) → Proceed(CarA) All three conditions are true.

Therefore: Proceed(CarA) is derived.

Complete Substitution Summary

Rule	Variable substitution	MGU
Rule 1	x = Ambulance	{x/Ambulance}
Rule 2	x = Ambulance	{x/Ambulance}
Rule 3	x = CarA, y = Ambulance	{x/CarA, y/Ambulance}

(b) Resolution Proof of Proceed(CarA) Step 1 – Convert Rule 1 into CNF Original:
Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) Using:
A → B ≡ ¬A ∨ B we obtain:
¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)

Step 2 – Convert Rule 2 Original:
ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) This gives two clauses:
¬ClearPath(x) ∨ GreenSignal(x)
¬ClearPath(x) ∨ ¬RedSignal(x)

Step 3 – Convert Rule 3 Original:
Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) CNF form:
¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)

Relevant Facts Vehicle(Ambulance) EmergencyType(Ambulance) Vehicle(CarA) Behind(CarA,Ambulance)

Resolution Step 1 Take:
¬Vehicle(Ambulance) ∨ ¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance) and:
Vehicle(Ambulance)

Resolving complementary literals gives:
¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)

Resolution Step 2 Resolve with:
EmergencyType(Ambulance) Therefore: ClearPath(Am Resolution Step 3
Use Rule 2:
¬ClearPath(Ambulance) ∨ GreenSignal(Ambulance) Resolving with:
ClearPath(Ambulance) gives: GreenSignal(Ambulance)

Resolution Step 4
Use Rule 3:
¬Vehicle(CarA) ∨ ¬Behind(CarA,Ambulance) ∨ ¬GreenSignal(Ambulance) ∨ Proceed(CarA)
Resolve with:
Vehicle(CarA) Result:
¬Behind(CarA,Ambulance) ∨ ¬GreenSignal(Ambulance) ∨ Proceed(CarA)
GreenSignal(Ambulance) ∨ Proceed(CarA)

Resolution Step 6 Resolve with: GreenSignal(Ambulance) Therefore: Proceed(CarA)

Resolution Chain Vehicle(Ambulance)
+
EmergencyType(Ambulance)
↓
ClearPath(Ambulance)
↓
GreenSignal(Ambulance)
↓
Vehicle(CarA) + Behind(CarA,Ambulance)
↓ Proceed(CarA) Hence:
Proceed(CarA) is logically proved.

Can the System Handle Two Simultaneous Emergency Vehicles?
The current knowledge base uses variables, so technically the same rules can be applied to multiple emergency vehicles.
For example, suppose the system receives: Vehicle(Ambulance1) EmergencyType(Ambulance1) Vehicle(Ambulance2) EmergencyType(Ambulance2)
Rule 1 can be applied separately:
ClearPath(Ambulance1) ClearPath(Ambulance2) Rule 2 can then derive:
GreenSignal(Ambulance1) GreenSignal(Ambulance2) However, there is a limitation.
The current knowledge base does not explain what should happen if both emergency vehicles require the same intersection or road at the same time.
For a practical system, additional information is needed. Required additional facts
Examples include:
At(Ambulance1,Intersection1) At(Ambulance2,Intersection1) Priority(Ambulance1,Ambulance2) SameIntersection(Ambulance1,Ambulance2) Possible additional rules

A priority rule could be:
HigherPriority(x,y) ∧ Conflict(x,y) → GiveWay(y) Another rule could identify a conflict:
SameIntersection(x,y) ∧ GreenSignal(x) ∧ GreenSignal(y) → Conflict(x,y)
Therefore, the existing KB can handle the basic reasoning for multiple emergency vehicles, but it is not sufficient for complete simultaneous traffic management.
It needs additional information about:

- vehicle priority
- intersection occupancy
- road conflicts
- current vehicle position
- signal coordination

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) **P1: SoilDry → IrrigationNeeded**
2) **P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod**
3) **P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk**
4) **Facts: SoilDry = True, CropWheat = True**

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justification:

CASE STUDY 3 – AGRICULTURAL EXPERT REASONING SYSTEM

Introduction

An agricultural advisory system helps farmers make decisions about irrigation. The system contains rules connecting soil conditions, crop type and irrigation methods.
The knowledge base contains:

P1: SoilDry → IrrigationNeeded
P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk Facts:
SoilDry CropWheat

(a) Conversion into CNF Meaning of CNF
Conjunctive Normal Form represents a logical expression as an AND of one or more clauses. The main conversion rule required here is:
A → B ≡ ¬A ∨ B

P1 Conversion Original:
SoilDry → IrrigationNeeded
Eliminate implication:
¬SoilDry ∨ IrrigationNeeded Therefore

P1 = ¬SoilDry ∨ IrrigationNeeded

P2 Conversion Original:
IrrigationNeeded ∧ CropWheat → ApplyDripMethod Using:
A → B ≡ ¬A ∨ B we get:
¬(IrrigationNeeded ∧ CropWheat) ∨ ApplyDripMethod Using De Morgan's law:
¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod Therefore:
P2 = ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod This is already in CNF.

P3 Conversion Original:
¬ApplyDripMethod ∧ CropWheat → CropAtRisk Eliminating implication:
¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk Using De Morgan's law:
ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk Therefore:
P3 = ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

Facts in CNF
The facts are already individual clauses: C4 = SoilDry
C5 = CropWheat

Final CNF Knowledge Base
Therefore, the complete CNF representation is: C1: ¬SoilDry ∨ IrrigationNeeded
C2: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod C3: ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk
C4: SoilDry
C5: CropWheat

Resolution Refutation for ApplyDripMethod Step 1 – State the goal
The theorem to prove is:
ApplyDripMethod
For Resolution Refutation, we assume the opposite of the goal. Therefore:

¬ApplyDripMethod
is added to the knowledge base. Let:
C6 = ¬ApplyDripMethod
The purpose is to derive the empty clause □.

Step 2 – Derive IrrigationNeeded Take:

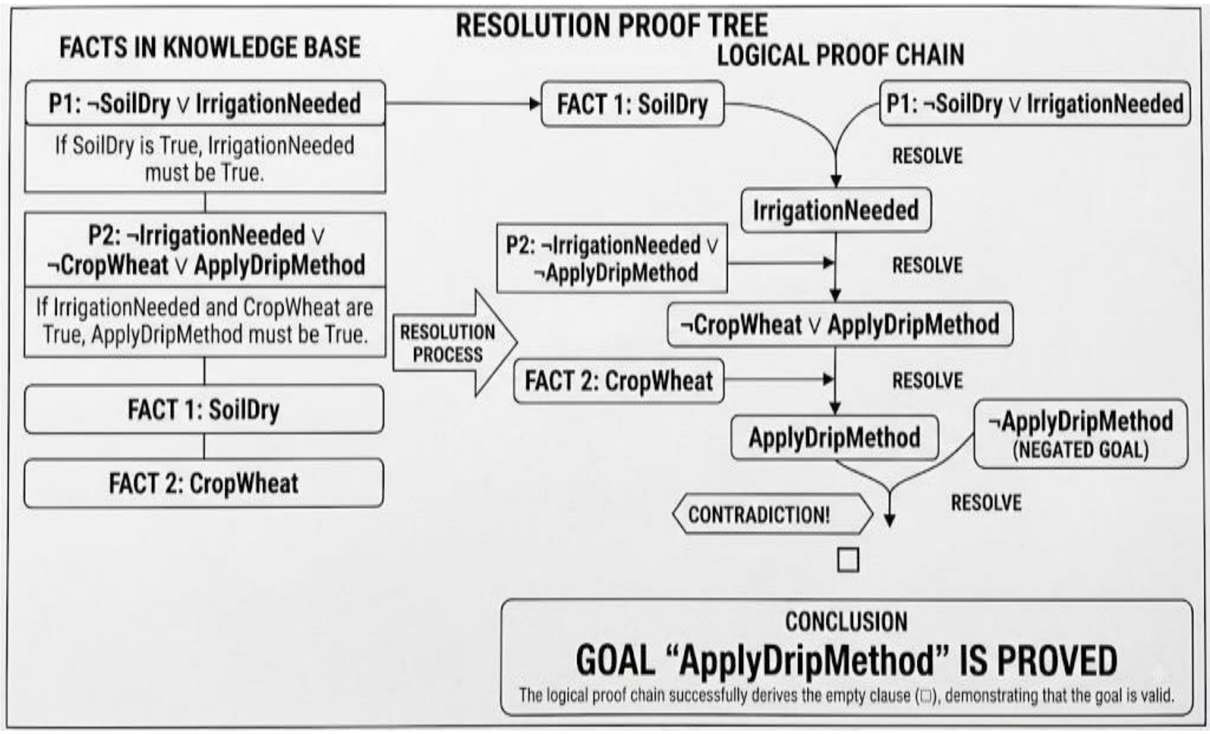
C1: ¬SoilDry ∨ IrrigationNeeded and:
C4: SoilDry
The complementary literals are:
¬SoilDry and SoilDry After resolution:
C7: IrrigationNeeded

Step 3 – Derive ApplyDripMethod Take:
C2: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod and:
C7: IrrigationNeeded Resolving them gives:
C8: ¬CropWheat ∨ ApplyDripMethod

Step 4 – Use the Wheat Fact We have:
C5: CropWheat Resolve C8 with C5:
¬CropWheat ∨ ApplyDripMethod and:
CropWheat Therefore:
C9: ApplyDripMethod

Step 5 – Contradiction We have:
C9: ApplyDripMethod and the negated goal:
C6: ¬ApplyDripMethod Resolving these gives:
□
The empty clause represents a contradiction. Therefore, our assumption that:
¬ApplyDripMethod is false.
Hence:
ApplyDripMethod is proved.

Resolution Proof Tree



Therefore:

KB ⊨ ApplyDripMethod

The symbol ⊨ means "logically entails."

(b) **Removing the SoilDry Fact Now consider what happens if:**

SoilDry

is removed from the knowledge base. The remaining fact is:

CropWheat The rule:

SoilDry → IrrigationNeeded

still exists, but its condition cannot be established. Therefore:

IrrigationNeeded cannot be derived.

Effect on ApplyDripMethod P2 requires:

IrrigationNeeded ∧ CropWheat

Although CropWheat is known, IrrigationNeeded is not known. Therefore:

ApplyDripMethod cannot be derived.

P3 is:

¬ApplyDripMethod ∧ CropWheat → CropAtRisk A common mistake would be to say:

"Since ApplyDripMethod cannot be proved, ¬ApplyDripMethod must be true." This is not correct in classical logic.

Not being able to prove a statement does not automatically prove its negation. We have:

ApplyDripMethod = not derivable but we do not have:

¬ApplyDripMethod = true Therefore, P3 cannot be fired. Final result after removing SoilDry

Statement	Result
SoilDry	Removed
IrrigationNeeded	Not derivable
ApplyDripMethod	Not derivable

¬ApplyDripMethod	Not given
CropAtRisk	Not derivable

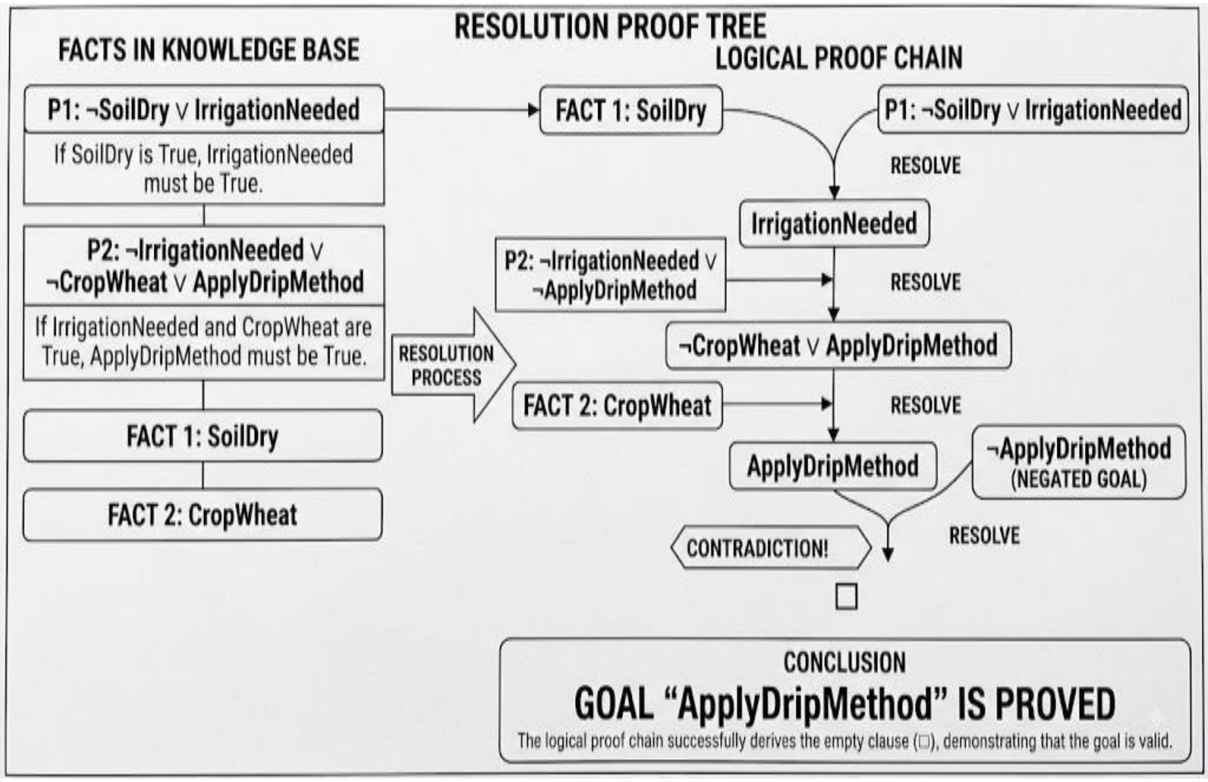
Therefore:
CropAtRisk does not follow from the remaining knowledge base.
This demonstrates the difference between lack of information and negative information.
C1: ¬SoilDry ∨ IrrigationNeeded and:
C4: SoilDry
The complementary literals are:
¬SoilDry and SoilDry After resolution:
C7: IrrigationNeeded

Step 3 – Derive ApplyDripMethod Take:
C2: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod and:
C7: IrrigationNeeded Resolving them gives:
C8: ¬CropWheat ∨ ApplyDripMethod

Step 4 – Use the Wheat Fact We have:
C5: CropWheat Resolve C8 with C5:
¬CropWheat ∨ ApplyDripMethod and:
CropWheat Therefore:
C9: ApplyDripMethod

Step 5 – Contradiction We have:
C9: ApplyDripMethod and the negated goal:
C6: ¬ApplyDripMethod Resolving these gives:
□
The empty clause represents a contradiction. Therefore, our assumption that:
¬ApplyDripMethod is false.
Hence:
ApplyDripMethod is proved.

Resolution Proof Tree



Therefore:

KB ⊨ ApplyDripMethod
The symbol ⊨ means "logically entails."

(c) **Removing the SoilDry Fact Now consider what happens if:**
SoilDry
is removed from the knowledge base. The remaining fact is:
CropWheat The rule:
SoilDry → IrrigationNeeded
still exists, but its condition cannot be established. Therefore:
IrrigationNeeded cannot be derived.

Effect on ApplyDripMethod P2 requires:
IrrigationNeeded ∧ CropWheat
Although CropWheat is known, IrrigationNeeded is not known. Therefore:
ApplyDripMethod cannot be derived.

P3 is:
¬ApplyDripMethod ∧ CropWheat → CropAtRisk A common mistake would be to say:
"Since ApplyDripMethod cannot be proved, ¬ApplyDripMethod must be true." This is not correct in classical logic.
Not being able to prove a statement does not automatically prove its negation. We have:
ApplyDripMethod = not derivable but we do not have:
¬ApplyDripMethod = true Therefore, P3 cannot be fired. Final result after removing SoilDry

Statement	Result
SoilDry	Removed
IrrigationNeeded	Not derivable
ApplyDripMethod	Not derivable
¬ApplyDripMethod	Not given
CropAtRisk	Not derivable

Conclusion:

Therefore:

CropAtRisk does not follow from the remaining knowledge base.

This demonstrates the difference between lack of information and negative information.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) **Stench [1,2] → Wumpus is adjacent to cell [1,2]**
2) **Breeze [1,1] → Pit is adjacent to cell [1,1]**
3) **Glitter [x,y] → Gold is at [x,y]**
4) **¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]**

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

CASE STUDY 4 – WUMPUS WORLD KNOWLEDGE AGENT

Introduction

Wumpus World is a classic AI problem in which an intelligent agent explores a grid environment. The agent cannot directly see all the dangers. Instead, it receives percepts such as Stench, Breeze and Glitter and uses logical rules to reason about the environment.

The agent receives:

- Stench at [1,2]

- Breeze at [1,1]
- Glitter at [2,2] The given rules are:

Rule 5:

Stench[1,2] → Wumpus is adjacent to [1,2]

Rule 6:

Breeze[1,1] → Pit is adjacent to [1,1]

Rule 7:
Glitter[x,y] → Gold is at [x,y]

- Breeze at [1,1]
- Glitter at [2,2] The given rules are:

Rule 5:
Stench[1,2] → Wumpus is adjacent to [1,2]
Rule 6:
Breeze[1,1] → Pit is adjacent to [1,1]
Rule 7:
Glitter[x,y] → Gold is at [x,y]

- Breeze at [1,1]
- Glitter at [2,2] The given rules are:

Rule 5:
Stench[1,2] → Wumpus is adjacent to [1,2]
Rule 6:
Breeze[1,1] → Pit is adjacent to [1,1]
Rule 7:
Glitter[x,y] → Gold is at [x,y]
.

Step 2 – Apply Rule 5
Rule 5:
Stench[1,2] → Wumpus adjacent to [1,2]
Since:
Stench[1,2] = True the rule fires.
The cells adjacent to [1,2] are:
[1,1], [1,3], [2,2]
Therefore, the agent concludes:
Wumpus ∈ {[1,1], [1,3], [2,2]}
This does not mean the Wumpus is definitely in all three cells. It means these are the possible locations based on the available information.

- Breeze at [1,1]
- Glitter at [2,2] The given rules are:

Rule 5:
Stench[1,2] → Wumpus is adjacent to [1,2]
Rule 6:
Breeze[1,1] → Pit is adjacent to [1,1]
Rule 7:
Glitter[x,y] → Gold is at [x,y]
.

Step 2 – Apply Rule 5.

- Breeze at [1,1]
- Glitter at [2,2] The given rules are:

Rule 5:
Stench[1,2] → Wumpus is adjacent to [1,2]
Rule 6:
Breeze[1,1] → Pit is adjacent to [1,1]
Rule 7:
Glitter[x,y] → Gold is at [x,y]
.

Step 2 – Apply Rule 5
Rule 5:
Stench[1,2] → Wumpus adjacent to [1,2]
Since:
Stench[1,2] = True the rule fires.
The cells adjacent to [1,2] are:
[1,1], [1,3], [2,2]
Therefore, the agent concludes:
Wumpus ∈ {[1,1], [1,3], [2,2]}
This does not mean the Wumpus is definitely in all three cells. It means these are the possible locations based on the available information.

Step 4 – Apply Rule 7
Rule 7:
Glitter[x,y] → Gold[x,y] The agent observes: Glitter[2,2]
Therefore:

Gold[2,2] is derived.
This gives the agent a direct target location.

Step 5 – Apply Rule 8
Rule 8 is:
¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]
To use this rule, the agent must know both:
¬Wumpus[x,y] and
¬Pit[x,y]
for a particular cell.
The given percepts do not provide enough information to establish these negative facts. Therefore, the agent cannot yet derive any definite:
Safe[x,y]
conclusion from Rule 8.

Belief State

The resulting belief state is:

Information	Logical conclusion
Stench[1,2]	Wumpus is adjacent to [1,2]
Breeze[1,1]	Pit is adjacent to [1,1]
Glitter[2,2]	Gold is at [2,2]
Wumpus candidates	[1,1], [1,3], [2,2]
Pit candidates	[1,2], [2,1]

(a) **Forward Chaining for Wumpus and Pit Locations**

Forward Chaining begins with the percepts and moves toward new conclusions.

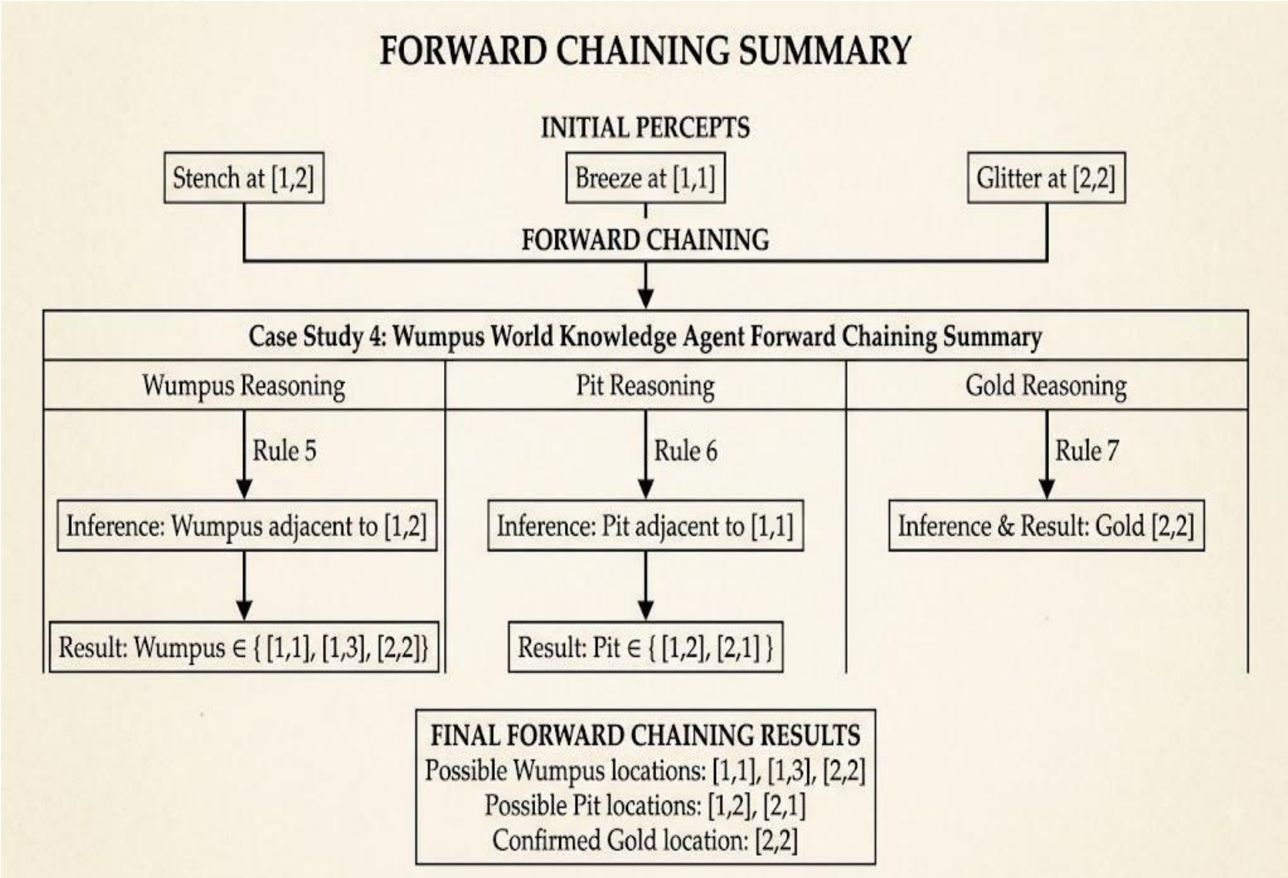
Wumpus Reasoning Initial fact:
Stench[1,2] Apply Rule 5:
Stench[1,2] → Wumpus adjacent to [1,2]
Adjacent cells:
[1,1], [1,3], [2,2]
Therefore:
Possible Wumpus locations = [1,1], [1,3], [2,2]

Pit Reasoning Initial fact:
Breeze[1,1] Apply Rule 6:
Breeze[1,1] → Pit adjacent to [1,1]
Adjacent cells:
[1,2], [2,1]
Therefore:
Possible Pit locations = [1,2], [2,1]

Gold Reasoning Initial fact:

Glitter[2,2] Apply Rule 7:
Glitter[2,2] → Gold[2,2]
Therefore: Gold is at [2,2].

Forward Chaining Diagram



(b) Is the Path [1,1] → [1,2] → [2,2] Safe?

The agent wants to follow:

[1,1] → [1,2] → [2,2]

to reach the gold.

The final destination [2,2] contains the gold because:

Glitter[2,2] → Gold[2,2]

However, the agent must determine whether the path can be confirmed safe.

Cell [1,1]

The agent starts at [1,1]. There is a:

Breeze[1,1]

This means that a pit is adjacent to [1,1]. Possible pit locations include:

[1,2] and [2,1]

Therefore, the next cell [1,2] is a possible pit location. So the agent cannot conclude that [1,2] is safe.

Cell [1,2] There is a:

Stench[1,2]

This means that a Wumpus is adjacent to [1,2]. The possible Wumpus locations include:

[1,1], [1,3], [2,2]

Final Decision

The agent should **not assume the path is safe**, because:

- [1,2] may contain a Pit.
- [2,2] may contain the Wumpus.
- The KB does not prove .
- The KB does not prove .
- Therefore, the Safe rule cannot used